Noise Schedule Ablation in Diffusion Models

Phase 2 Report

Group 38
Foundation Models and Generative AI
Indian Institute of Technology Jodhpur

Item	Value
Image-to-image task	Visible-light photograph to infrared thermal image
Pretrained model	timbrooks/instruct-pix2pix (InstructPix2Pix)
Dataset	LLVIP, 2,160 aligned pairs used, at 512 x 512
Native noise parameter	alpha-bar (cumulative signal retention), theta_t in (0, 1)
Declared exponent x	2.35
Random seed	20261026
Hardware	NVIDIA RTX 5050 Laptop GPU, 8 GB
Total compute time	206 minutes for the full pipeline

1. Task and Model

1.1 The task

We translate a visible-light photograph into the infrared thermal image of the same scene. The input is an ordinary colour photograph taken by a normal camera at night. The output is the greyscale image that a thermal camera, sitting next to that normal camera and pointed at exactly the same place at exactly the same moment, recorded. In a thermal image brightness means temperature, not reflected light, so a person shows up bright even in complete darkness, and a leafy tree shows up dark even if it is lit by a street lamp.

Both images are 512 x 512 pixels. The same instruction is given to the model for every image: "turn this visible-light photo into an infrared thermal image". The instruction never changes, so the only thing that varies between examples is the picture itself.

We picked this task because the correct answer is known. The dataset gives us the real thermal photograph for every visible photograph, pixel for pixel, so we can compare what the model produced against what the thermal camera actually recorded. Tasks like toonification have no single correct answer and would have left us with nothing to measure.

1.2 The model

We used InstructPix2Pix, published by Brooks, Holynski and Efros (CVPR 2023). It is a latent diffusion model built on top of Stable Diffusion v1.5, and it takes an image plus a written instruction and returns an edited image.

It works in three parts. A VAE encoder squeezes the 512 x 512 x 3 image down to a 64 x 64 x 4 array, called a latent. All the diffusion happens on that small array, which is what makes it fit in 8 GB of memory. A CLIP text encoder turns the instruction into a vector. A UNet of 862,721,220 parameters does the actual denoising. The UNet's first convolution takes 8 channels, not the usual 4: four for the noisy latent it is cleaning up, and four for the latent of the input image it is supposed to be editing. That second group of four is how the input photograph is fed in as a condition.

1.3 The forward process and the symbols

Diffusion training works by deliberately damaging a clean image and asking the model to say what the damage was. The damage is added in T = 1000 steps. At step t a little Gaussian noise is mixed into whatever came out of step t - 1:

$$x_t = \mathit{sqrt}(1 - \mathit{beta_t}) \cdot x_{t-1} + \mathit{sqrt}(\mathit{beta_t}) \cdot \mathit{epsilon}, \quad \mathit{epsilon} \sim N(0, I)$$

Doing that a thousand times in a row would be slow, so the standard trick is to collapse all t steps into one. Writing alpha_t = 1 - beta_t and alpha-bar_t = alpha_1 . alpha_2 . ... . alpha_t, the clean image can be jumped to any timestep directly:

$$x_t = \mathit{sqrt}(\mathit{alpha\text{-}bar_t}) \cdot x_0 + \mathit{sqrt}(1 - \mathit{alpha\text{-}bar_t}) \cdot \mathit{epsilon}$$

Symbol	Meaning
x_0	the clean latent of the target thermal image
x_t	the latent after t steps of noise have been added
epsilon	the Gaussian noise that was added, mean 0 and variance 1
t	the timestep, from 1 to T
T	the number of diffusion timesteps, 1000 for this model
beta_t	how much fresh noise step t adds; runs from 0.00085 to 0.012

Symbol	Meaning
alpha_t	1 - beta_t, the fraction surviving one single step
alpha-bar_t	the product of alpha_1 to alpha_t, the fraction of the original image still present at step t
x	our declared exponent, 2.35

1.4 The native noise parameterisation

We declare theta_t = alpha-bar_t.

There are three candidates in this model: beta_t, alpha_t and alpha-bar_t. All three stay inside (0, 1), so any of them is allowed by the rules of the assignment. We chose alpha-bar for two reasons.

- It is the quantity the training loop actually reads. The diffusers DDPMScheduler keeps an array called alphas_cumprod and the line that corrupts the image indexes straight into it. beta is only used once, at setup, to build that array. If we want to change what training sees, alpha-bar is the thing to change.
- Raising alpha-bar to a power is the same as raising every single-step alpha to the same power, because (alpha_1 . alpha_2 . ... . alpha_t)^x = alpha_1^x . alpha_2^x . ... . alpha_t^x. So the modified schedule is still a valid product of per-step survival factors. It is a genuine reparameterisation of the forward process, not an arbitrary edit of one number.

We also checked what beta would have given us before deciding. With theta = beta and an exponent in the allowed range, alpha-bar at t = 1000 comes out at about 0.96, meaning 96 percent of the image would still be there at the point where the process is supposed to have destroyed it. Run B would have been trained on a forward process that never actually adds noise, and the whole comparison would have been meaningless. Picking alpha-bar avoided that.

1.5 Links

Resource	Link
Paper	arXiv:2211.09800 - InstructPix2Pix: Learning to Follow Image Editing Instructions
Official code	https://github.com/timothybrooks/instruct-pix2pix
Pretrained weights	https://huggingface.co/timbrooks/instruct-pix2pix
Training script we started from	huggingface/diffusers, tag v0.31.0, examples/instruct_pix2pix/train_instruct_pix2pix.py

2. Dataset and Verification

2.1 The dataset

We used LLVIP (Low-Light Visible-Infrared Paired dataset), released in 2021 by Beijing University of Posts and Telecommunications. It is a street surveillance dataset recorded at night with a visible camera and a thermal camera mounted together, so every visible frame has a thermal frame that lines up with it pixel for pixel.

Property	Value
Total pairs available	15,488
Pairs we used	2,160 (2,000 train, 60 validation, 100 test)
Original resolution	1280 x 1024
Resolution we worked at	512 x 512, centre-cropped then resized
Capture scenes	26, identified by the first two digits of each filename
Content	night-time pedestrian street scenes
Source	https://github.com/bupt-ai-cz/LLVIP
Paper	arXiv:2108.10831 (ICCV 2021 Workshops)
Licence	free for non-commercial research use

It suits this task for three reasons. The pairs are registered, so there is a correct answer for every input and we can score the output instead of guessing whether it looks right. The images are already dark and low-contrast, which is exactly the situation where a thermal camera is worth having, so the task is a real one. And a thermal image is a genuinely different kind of picture, not a filter, so the model has to learn something rather than adjust colours.

We did not use all 15,488 pairs. On one 8 GB laptop GPU, two full training runs plus three inference passes had to finish in a few hours, and 2,000 training pairs over 5 epochs was what fitted. The split is described in section 4.

2.2 Evidence that LLVIP was not in the pretraining data

We use route (b) from the declaration: a citation describing the pretraining data domain, plus an argument that LLVIP is a clearly different domain.

The citation

InstructPix2Pix, arXiv:2211.09800, Section 3.1.1 "Generating Instructions and Paired Captions", page 3. The paper states that "the input captions are real image captions from LAION-Aesthetics". Section 3.1 on the same page states that the paired training images are produced by "a large language model and a text-to-image model", and the Figure 2 caption on page 3 states "We then use StableDiffusion in combination with Prompt-to-Prompt" to generate the before and after images.

The argument

There are two separate reasons LLVIP cannot be in there, and either one on its own is enough.

First, every training image InstructPix2Pix saw is synthetic. The paper does not collect real before-and-after photo pairs. It writes caption pairs with a language model and then renders both images with Stable Diffusion. So no real photograph from any dataset at all appears in its fine-tuning set, including LLVIP's.

Second, the captions that seed those images come from LAION-Aesthetics, which is a subset of LAION-5B filtered by an aesthetic scoring model. LAION-5B is built by scraping image-and-alt-text pairs from Common Crawl. LLVIP is distributed as a zip archive from a GitHub repository under a research

licence. It has no captions and no alt text at all, only numbered image files and bounding-box labels, so there is nothing in it for a caption-based pipeline to pick up. It was also filtered for aesthetic appeal, and a fixed-angle night surveillance frame is the opposite of what that filter keeps.

On domain: LAION-Aesthetics is aesthetically scored colour photography and artwork from the open web. LLVIP is registered thermal infrared radiometry, captured from a fixed overhead surveillance mount, at night, with a second sensor measuring emitted heat rather than reflected light. The thermal half of every pair is not a photograph in the ordinary sense at all. Nothing in the base model's training has the brightness-equals-temperature convention, which is the main thing our fine-tuning has to teach it.

3. Schedule Analysis

3.1 Confirming theta_t stays in (0, 1)

This has to hold for both runs, otherwise raising it to a power is not meaningful. We checked it twice: once by argument and once by measuring the actual arrays.

The argument. The model uses a scaled-linear beta schedule running from 0.00085 to 0.012, so every beta_t is strictly between 0 and 1. Then alpha_t = 1 - beta_t is also strictly between 0 and 1. alpha-bar_t is the product of t such numbers, and a product of numbers in (0, 1) is itself in (0, 1), never reaching either end. It is also strictly decreasing, because each extra factor is less than 1. Finally, raising any number in (0, 1) to a positive power leaves it in (0, 1), so the modified schedule inherits the property automatically.

The measurement. scripts/schedule.py walks all 1000 values of both arrays and asserts the bounds. The same assertion runs again at the start of every training run and every inference run, so nothing can slip through unnoticed. The measured extremes are:

Quantity	Run A (original)	Run B (powered, x = 2.35)
smallest alpha-bar, at t = 1000	0.004660095	0.000003316864
largest alpha-bar, at t = 1	0.999150	0.998004
strictly inside (0, 1) for all t	yes, all 1000 values	yes, all 1000 values

3.2 The two schedules plotted together

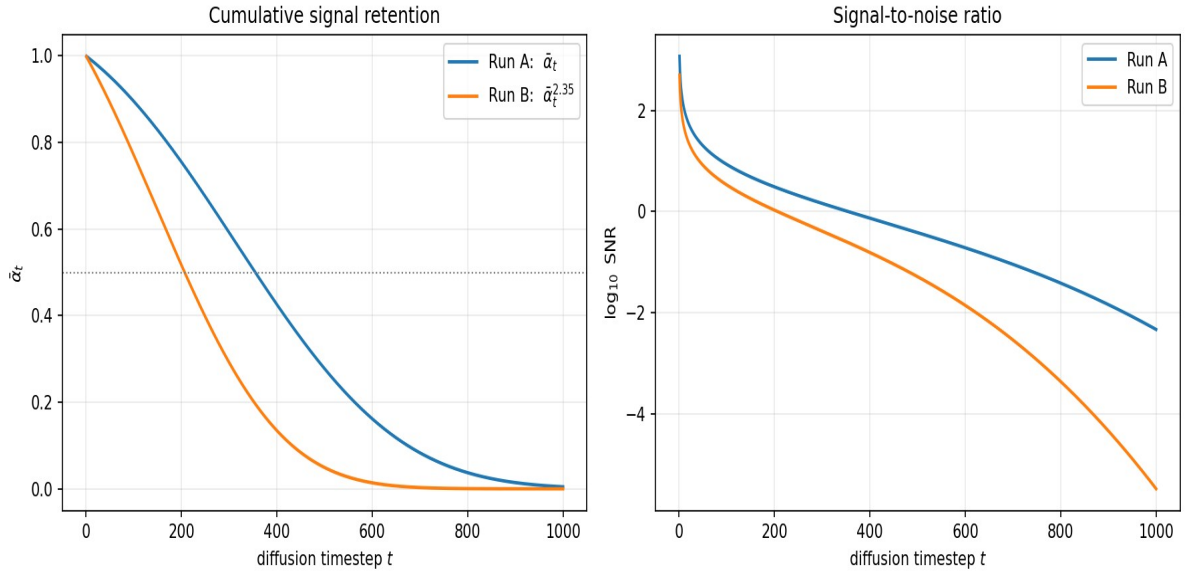


Figure 1. Left: alpha-bar against t for both runs, with the half-way line marked. Right: the same two schedules as log-10 signal-to-noise ratio.

t	alpha-bar (A)	sqrt(alpha-bar) (A)	alpha-bar (B)	sqrt(alpha-bar) (B)
1	0.999150	0.999575	0.998004	0.999001
250	0.675432	0.821847	0.397665	0.630607
500	0.277669	0.526943	0.049237	0.221894
750	0.056623	0.237957	0.001174	0.034259

t	alpha-bar (A)	sqrt(alpha-bar) (A)	alpha-bar (B)	sqrt(alpha-bar) (B)
1000	0.004660	0.068265	0.0000033	0.001821

Table 1. alpha-bar and its square root at five timesteps. The square root is the number that actually multiplies the clean image in the forward equation, so it is the honest measure of how much of the picture is left.

3.3 What changes in the forward process when x > 1

Because alpha-bar is always between 0 and 1, raising it to 2.35 always makes it smaller. The only exceptions are the two endpoints, 0 and 1, which stay put. So the orange curve in Figure 1 sits under the blue one everywhere in between, and the gap is widest in the middle.

Putting that back into the forward equation, x_t = sqrt(alpha-bar_t) . x_0 + sqrt(1 - alpha-bar_t) . epsilon, a smaller alpha-bar means a smaller multiplier on the clean image and a larger one on the noise. Concretely, at t = 500 Run A still has 52.7 percent of the image amplitude left, while Run B has 22.2 percent. At t = 750 it is 23.8 percent against 3.4 percent.

The clearest single way to say it: the point where exactly half the signal is gone moves from t = 355 under the original schedule to t = 207 under the powered one. Corruption happens 148 steps earlier. The forward process is front-loaded. Most of the destruction now takes place in the first fifth of the timeline, and by the time t reaches 600 there is effectively nothing left to destroy.

There is a second effect at the far end. Under the original schedule alpha-bar never actually reaches zero; sqrt(alpha-bar) at t = 1000 is 0.068, so about 7 percent of the image amplitude is still sitting there at the timestep that is supposed to be pure noise. This is a known quirk of Stable Diffusion's schedule and it means the model is never trained on a truly blank input. Raising the schedule to 2.35 drops that leftover to 0.0018, which is 0.18 percent. Run B's last timestep is, for practical purposes, genuinely pure noise.

3.4 Effect on the signal-to-noise ratio

Signal-to-noise ratio at a timestep is the power of what is left of the image divided by the power of the noise:

$$SNR(t) = \text{alpha-bar}_t / (1 - \text{alpha-bar}_t)$$

Substituting the powered schedule gives SNR_B(t) = alpha-bar_t^x / (1 - alpha-bar_t^x). Taking logs is what makes the relationship clear:

$$\log SNR_B(t) = x \cdot \log(\text{alpha-bar}_t) - \log(1 - \text{alpha-bar}_t^x)$$

Once alpha-bar is small, which is most of the timeline, the second term is log(1 - something tiny), which is almost exactly zero. What is left is log SNR_B approximately equal to x times log SNR_A. The exponent stretches the log-SNR curve by a constant factor in the low-signal region.

The numbers confirm this exactly. At t = 1000, log-10 SNR for Run A is -2.330. Multiplying by 2.35 gives -5.476. The measured value for Run B is -5.479. The difference is rounding. This is the cleanest piece of evidence in the report that the transform did what the mathematics says it should.

t	log10 SNR (A)	log10 SNR (B)	ratio B / A
1	+3.070	+2.699	0.88
250	+0.318	-0.180	not meaningful (A near zero)
500	-0.415	-1.286	3.10
750	-1.222	-2.930	2.40

t	log10 SNR (A)	log10 SNR (B)	ratio B / A
1000	-2.330	-5.479	2.35

Table 2. The ratio approaches exactly 2.35 as alpha-bar gets small, which is where the approximation above becomes exact. Near t = 1 the log-SNR is positive and large, the approximation does not apply, and the exponent compresses rather than stretches.

The practical consequence is a shift in where the model spends its training effort. Training picks a timestep at random for every example. Under Run A, a random timestep is fairly likely to land somewhere with a moderate amount of signal left, where the job is to sharpen and refine. Under Run B, far more of the timestep range sits in the very low signal region, where the job is to invent structure from almost nothing. Run B therefore spends more of its budget learning overall layout and brightness, and less on fine texture. Section 7 shows that this is what we actually observe in the images.

4. Experimental Setup

4.1 Dataset preparation

LLVIP was downloaded as a zip from the official GitHub repository and unpacked so that the visible and infrared folders sat side by side. The two folders use identical filenames, which is how a visible frame is matched to its thermal partner.

Each image was centre-cropped to a square and then resized to 512 x 512 with Lanczos resampling, and written to a cache folder as JPEG at quality 95. Both halves of a pair get exactly the same crop and resize, so the pixel alignment that makes the dataset useful is preserved. At load time the pixels are scaled from 0-255 to the range -1 to 1, which is what the VAE expects.

No augmentation was used. Random flips were available and switched off on purpose: with augmentation on, the two runs would see different random transforms and the comparison would no longer be a single-variable one. Every image both runs see is byte-identical.

The preparation script finishes with a guard that opens every file named in the split manifests and asserts it really is 512 x 512, so a half-finished cache from an interrupted run cannot quietly poison a training run.

4.2 Train, validation and test split

This needed care. LLVIP images are video frames, not independent photographs. The first two digits of a filename are the capture session, and consecutive frames in a session are nearly the same picture. A random split would put a frame and its near-duplicate on opposite sides of the train and test boundary, and the test score would then be measuring memorisation.

We measured how bad this would be. Two frames 596 apart inside session 01 differ by a mean absolute grey level of 15.22. Frames taken from two different sessions differ by between 17.78 and 46.87. So even the most distant pair within one session is more alike than the closest pair across sessions.

Every split boundary therefore falls on a session boundary, and within a session frames are taken at a stride rather than consecutively.

Split	Pairs	Sessions	Purpose
Train	2,000	01 to 16	the only data either run is trained on
Validation	60	17, 18, 25	checkpoint samples during training
Test	100	19 to 24 and 26	final scoring, used once per run

Sessions 19 to 24 and 26 are LLVIP's own official test sessions, so our test set is a subset of theirs and does not overlap the authors' training data either. The split is computed once, with seed 20261026, and written to data/train.txt, data/val.txt and data/test.txt. Both runs read those three files. Neither run recomputes the split, so there is no way for them to disagree about it.

4.3 Fine-tuning hyperparameters

Every value in this table is identical for Run A and Run B.

Setting	Value
Pretrained checkpoint	timbrooks/instruct-pix2pix (both runs start from this, Run B never starts from Run A)
Trainable parameters	LoRA adapter only: 3,188,736 of 862,721,220 (0.37 percent)

Setting	Value
LoRA rank / alpha	16 / 16
LoRA target modules	to_k, to_q, to_v, to_out.0 (the attention projections)
Optimizer	AdamW, betas 0.9 and 0.999, weight decay 0.01, eps 1e-8
Learning rate	1e-4, constant, no warmup
Batch size	4 (gradient accumulation 1, so effective batch 4)
Epochs	5
Total optimizer steps	2,500 (2,000 pairs / 4 = 500 steps per epoch)
Checkpoint interval	every 500 steps, which lands exactly on each epoch boundary
Precision	fp16 mixed precision
Gradient checkpointing	on
Resolution	512 x 512, centre crop
Dataloader workers	0 (deterministic ordering)
Seed	20261026, applied to Python, NumPy, torch and the dataloader generator
Only difference between A and B	the noise schedule

LoRA was used rather than full fine-tuning because the full UNet does not fit in 8 GB. Measured peak memory for our configuration at 512 x 512 and batch 4 was 2.51 GiB, at about 0.82 seconds per step in the benchmark and 2.1 seconds per step in real training, the difference being the VAE encoding of eight 512 x 512 images on every step. Each run took about 88 minutes. The LoRA configuration being identical in both runs is what keeps the comparison honest.

4.4 Validation during fine-tuning

Validation was qualitative and ran every 500 steps, which is once per epoch. At each of the five checkpoints the same four validation images were generated, with the same seed and the same number of sampler steps. Because the images and the seed are fixed, anything that looks different between two checkpoint folders is the model changing, not the sampler drawing different noise.

We deliberately did not use a scalar validation metric to choose a checkpoint, because doing so would have let the metric pick the winner before we had looked at the images. Section 6.5 shows why that would have been a mistake.

4.5 How the test set was used

The 100 test pairs were held out completely during training. They were used once per run, at the very end, with the final adapter from step 2500. The generated image was compared against the real thermal image using SSIM, PSNR, mean luminance error and contrast ratio. The same 100 inputs, the same seed and the same sampler settings were used for all three runs, so the three columns of the results table are directly comparable.

5. Implementation

5.1 Which files were modified

Only one file in the project is derived from someone else's code: scripts/finetune.py, which comes from the Hugging Face diffusers InstructPix2Pix training example at tag v0.31.0. Everything else in scripts/ we wrote ourselves, so there is nothing in those files to mark as modified.

We did not edit the example by hand. scripts/make_finetune.py takes the untouched baseline and applies fifteen patches, each anchored to a specific piece of text that must appear exactly once, and the script fails loudly if any anchor is missing or ambiguous. The generated file is scripts/finetune.py. This was done so the change set is reproducible rather than something that drifted over a week of editing, and so a reader can regenerate it and get the same file. Editing finetune.py directly has no effect, because the next regeneration overwrites it.

Quantity	Value
Baseline file	baseline/train_instruct_pix2pix_ORIGINAL.py (1,000 lines, unmodified)
Generated file	scripts/finetune.py (1,031 lines)
Lines marked # ADDED	104
Lines marked # MODIFIED	12
Total changed lines in the diff	309 (149 added, 160 removed)
Full line-by-line diff	report/code_diff.txt

The exponent is declared as a named constant at the top of scripts/schedule.py, line 15:

```
POWER_EXPONENT = 2.35  # ADDED - declared exponent, Phase 1 item 6
```

scripts/finetune.py imports it at line 53 rather than keeping a second copy, so there is only one place in the whole project where the number 2.35 is written down.

5.2 Original and modified lines, side by side

Four of the twelve modified sites are shown below. These are the ones that matter for the experiment. The rest are housekeeping (a removed import, logging lines, checkpoint saving) and are in report/code_diff.txt.

Change 1 - the noise schedule hook. This is the whole experiment.

ORIGINAL - baseline, line 490

```
noise_scheduler = DDPMScheduler.from_pretrained(
    args.pretrained_model_name_or_path,
    subfolder="scheduler")
```

MODIFIED - finetune.py, lines 541 to 547

```
noise_scheduler = DDPMScheduler.from_pretrained(
    args.pretrained_model_name_or_path,
    subfolder="scheduler")
verify_bounds(noise_scheduler, ...)  # ADDED
if args.run_mode == "B":  # ADDED
    apply_power_transform(noise_scheduler,
        POWER_EXPONENT)  # ADDED
    verify_bounds(noise_scheduler, ...)  # ADDED
_schedule_digest = log_schedule(...)  # ADDED
logger.info(f"checksum {_schedule_digest}")  # ADDED
```

Run A takes the "if" branch false and trains on the schedule the model shipped with. Run B takes it true. apply_power_transform computes alpha-bar to the power 2.35, then rebuilds the per-step alphas and betas from it so every quantity inside the scheduler stays consistent with the new alpha-bar. Both branches then write the full 1000-value array to disk and print its checksum.

Change 2 - the seed is compulsory.

ORIGINAL - baseline, line 223	MODIFIED - finetune.py, lines 268 to 271
parser.add_argument("--seed", type=int, default=None, help="A seed for reproducible training.")	parser.add_argument(# MODIFIED - was default=None "--seed", type=int, required=True, help="A seed for reproducible training.",)

The baseline allows the seed to be left out, in which case every run gets different randomness. An ablation whose two arms do not share a seed is not an ablation, so we made it compulsory. The run simply refuses to start without one.

Change 3 - do not widen a UNet that is already wide.

ORIGINAL - baseline, lines 510 to 520	MODIFIED - finetune.py, lines 566 to 582

in_channels = 8
out_channels = unet.conv_in.out_channels
unet.register_to_config(in_channels=in_channels)
with torch.no_grad():
  new_conv_in = nn.Conv2d(in_channels, ...)
  new_conv_in.weight.zero_()
  new_conv_in.weight[:, :4].copy_(
  unet.conv_in.weight)
  unet.conv_in = new_conv_in
	# MODIFIED - the widening below assumes a plain
	# Stable Diffusion UNet. instruct-pix2pix already
	# has 8 input channels, so I skip it.
	if unet.conv_in.in_channels == 8: # ADDED
	  logger.info("already 8 channels") # ADDED
	else: # ADDED
	  in_channels = 8
	  ... (baseline code unchanged inside)

The example assumes you are starting from plain Stable Diffusion, which has 4 input channels, and glues on 4 more zeroed channels for the conditioning image. We start from the already-trained InstructPix2Pix checkpoint, which has all 8. Running the widening on it threw away the trained conditioning weights and the run crashed with a tensor size mismatch of 4 against 8. Making it conditional fixed it.

Change 4 - only the adapter is optimised.

ORIGINAL - baseline, lines 610 to 611	MODIFIED - finetune.py, lines 690 to 691

optimizer = optimizer_cls(
  unet.parameters(),
  lr=args.learning_rate, ...)
	optimizer = optimizer_cls(
	  lora_params, # MODIFIED - was unet.parameters()
	  lr=args.learning_rate, ...)

Handing the optimizer all 862 million parameters would allocate Adam state for all of them and exhaust 8 GB immediately. lora_params is the 3,188,736 adapter parameters.

5.3 How we verified Run B actually used the powered schedule

This is the one thing in the project that, if it silently went wrong, would leave us with a report full of confident conclusions about nothing. So we checked it three different ways.

Checksums

Every piece of code that touches noise takes a SHA-256 of the full 1000-value alpha-bar array it is about to use, truncates it to 16 hex characters, and prints it. The expected values were recorded by scripts/schedule.py before any training started:

Run	Expected checksum	Observed in every stage
Run A	40873f7067e7f82d	40873f7067e7f82d
Run B	6fa947b412c6432e	6fa947b412c6432e

Six places print it: training for A, training for B, final inference for A, final inference for B, the checkpoint sampling, and the trajectory capture. All six matched. Run A and Run B print different values, which is the direct evidence that the two runs did not train on the same schedule. If they had printed the same value, the transform was not being applied and nothing downstream would be worth reading.

Bounds printed at every start

Every run prints the minimum and maximum of its alpha-bar array before it begins. Run A reports min 4.660095e-03 and max 0.999150. Run B reports min 3.316864e-06 and max 0.998004. Those are visibly different numbers in the console log, so a reader does not even need the checksum to see that something changed.

A guard on the commands themselves

main.py builds the two training commands and, before running anything, compares them token by token. It allows exactly three differences: the run mode letter, the output directory and the loss CSV path. If anything else has crept in, a different learning rate, a different batch size, a stray flag, it refuses to start. This is what stops the two arms drifting apart through a careless edit.

6. Results

6.1 Run 0 - the pretrained model with no fine-tuning

Run 0 applies timbrooks/instruct-pix2pix straight to our test images with the same instruction and the same seed, and no training at all.



Figure 2. Run 0. Left: the visible-light input. Middle: what the pretrained model produced. Right: the real thermal image.

The model does not do the task. It understands "infrared" as a colour-grading instruction: it desaturates, brightens and raises contrast, and leaves the photograph otherwise intact. The traffic light in rows 2 and 4 is still a coloured traffic light. In the real thermal image it is a dark rectangle, because an LED traffic light is cold. Row 3 is the clearest failure, where instead of converting the scene it has replaced the dark trees with pink blossom and discarded the car and the cyclist entirely.

This is the correct starting point for the experiment. It shows the pretrained model has no idea that brightness in a thermal image means temperature, which is the thing the fine-tuning has to teach it.

6.2 Fine-tuning loss curves

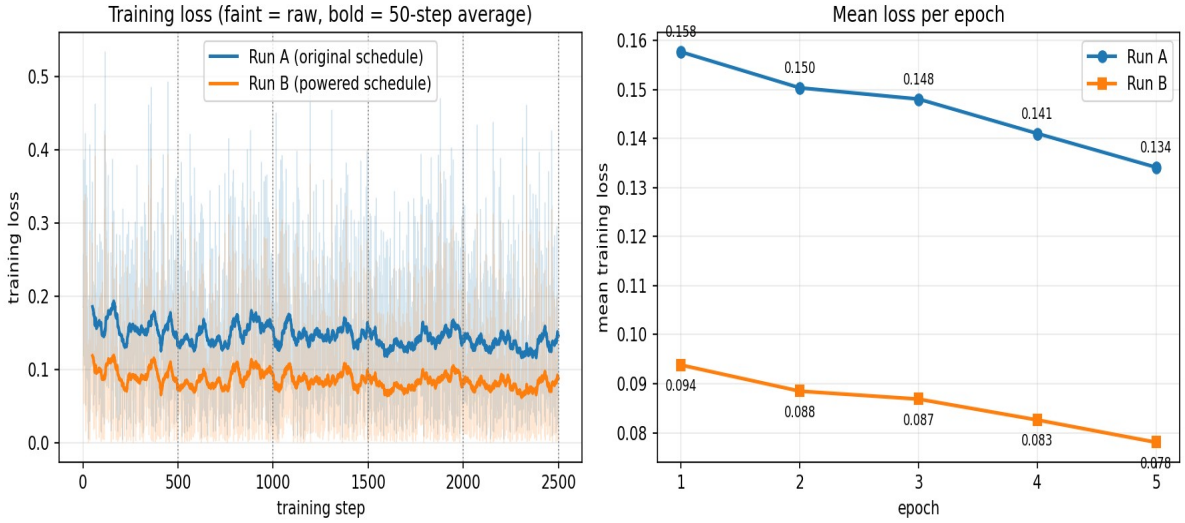


Figure 3. Left: per-step training loss for both runs, faint lines raw and bold lines a 50-step moving average, with epoch boundaries marked. Right: the mean loss per epoch, which is the data in the two submitted CSV files.

Epoch	Run A train_loss	Run B train_loss
1	0.157666	0.093796
2	0.150342	0.088471
3	0.148050	0.086850
4	0.141024	0.082589
5	0.134128	0.078033

Table 3. The contents of logs/runA_epoch_loss.csv and logs/runB_epoch_loss.csv.

Both curves go down steadily and neither shows instability. Run B's loss is lower than Run A's at every single step. Section 7 explains why that is not evidence that Run B learned better, and why these two numbers should not be compared with each other at all.

6.3 Samples at the five fine-tuning checkpoints

The same three validation images at each of the five epoch boundaries, same seed throughout.

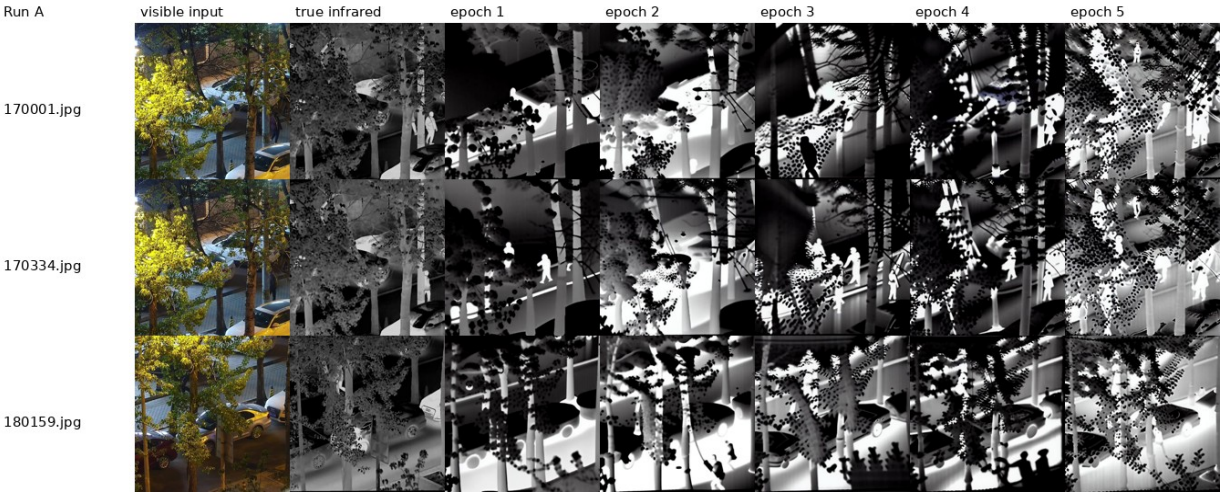


Figure 4. Run A, original schedule. Columns left to right: visible input, real thermal, then epochs 1 to 5.



Figure 5. Run B, powered schedule, x = 2.35. Same images, same seed, same columns.

6.4 Inference trajectories

For three test images we saved the noisy sample x_t at five equally spaced points of the 50-step reverse process, for both runs. The sampler steps captured were 0, 12, 24, 37 and 49, which correspond to diffusion timesteps t = 999, 754, 509, 244 and 0.

x_t is a latent, not an image, so each capture is decoded through the VAE before being written out. One practical note belongs here. At the earliest timesteps the latents are far larger than anything the VAE decoder was ever trained on, and decoding Run B's first capture directly overflowed and wrote a solid black frame. The capture script therefore divides each latent by its own standard deviation before decoding, and records that divisor in _trajectory_info.json so nothing is hidden. The divisors are themselves a measurement:

Capture	Run A divisor	Run B divisor
t = 999	12.9	410.5
t = 754	4.3	25.3
t = 509	2.3	4.5
t = 244	1.6	1.7
t = 0	1.0 (none needed)	1.0 (none needed)

Table 4. How far each run's latent sits outside the range the decoder expects, at each capture. A divisor of 1.0 means the latent was already in range.

At the first capture Run B's latent is 32 times further out of range than Run A's. This follows directly from the schedule: the sampler's starting noise scale is the reciprocal square root of alpha-bar, and alpha-bar at t = 999 is 0.004660 for Run A against 0.0000033 for Run B, which predicts a ratio of about 38. The two columns converge by t = 244 and are identical at t = 0, which is what they should do, because both runs end at the same place.

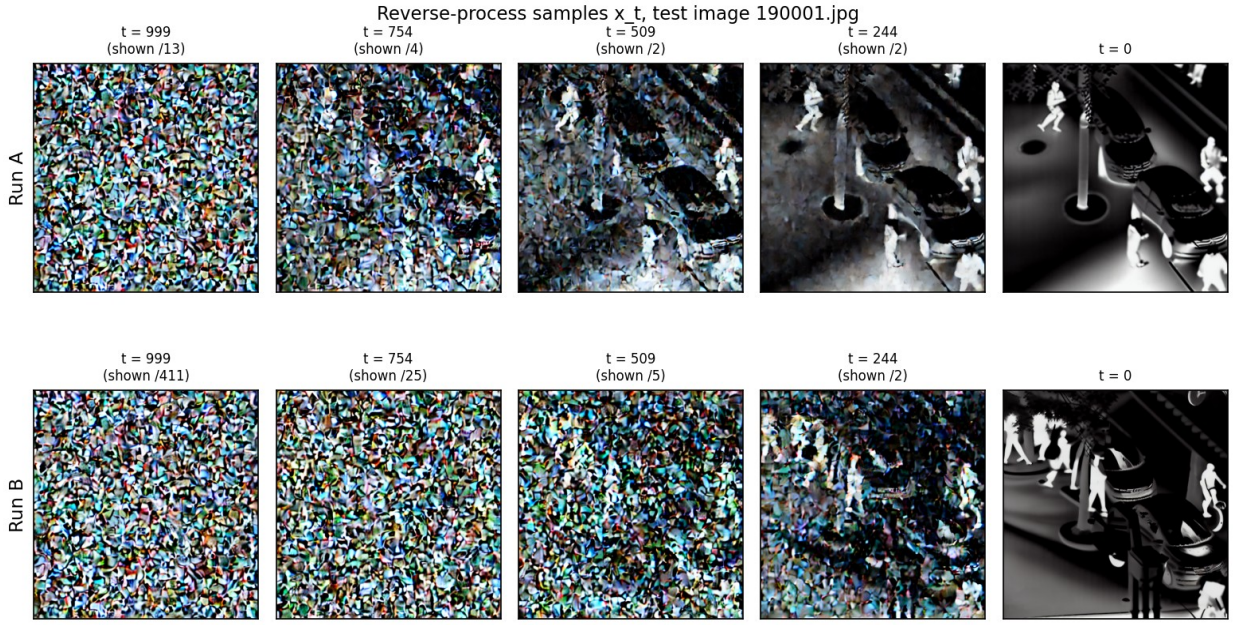


Figure 6. The reverse process for test image 190001.jpg. Top row Run A, bottom row Run B, same five timesteps, same seed. The divisor used to bring each frame into the decoder's range is printed above it.

The two rows show the schedule difference more plainly than any number in this report. At t = 509, exactly half way through sampling, Run A already has the scene: two cars, the lamp post, four pedestrians, all recognisable through the noise. Run B at the same timestep is still almost entirely noise, with only the faintest hint of the bright figures. Run B does not produce a readable image until t = 244. This is the front-loading from section 3.3 seen from the other direction. Run B's forward process destroyed the image much earlier, so its reverse process has much further to travel and cannot commit to structure until late. Both rows finish at t = 0 with a complete thermal image, and Run B's final frame has the cleaner figures and the better separated vehicles, which is the same thing Figure 7 shows on the test set.

6.5 Test-set scores

All three runs were scored on the same 100 held-out test pairs.

Run	SSIM	PSNR (dB)	Luminance error	Contrast ratio
visible input, no model	0.2591	-	-	-
Run 0 (no fine-tuning)	0.2954	11.86	-11.61	0.905
Run A (original schedule)	0.2829	8.45	+11.17	1.623
Run B (powered schedule)	0.2579	9.13	-24.99	1.357

Table 5. Test-set scores, 100 images each. Standard deviations of SSIM are 0.123, 0.063 and 0.062 for Runs 0, A and B. Luminance error is the mean of predicted minus true grey level, where 0 is perfect. Contrast ratio is the mean of predicted standard deviation over true standard deviation, where 1.0 is perfect. The first row is a control described below.

Why SSIM is our chosen metric: the output is supposed to reproduce the spatial structure of a scene that the input already contains, so a metric built on local means, variances and covariances tells us whether the structure survived the translation, which pixel-wise error does not.

The first row of Table 5 is a control we ran after seeing the results. It is the SSIM of the raw visible-light input scored against the thermal ground truth, with no model involved at all. It is reported here because the rest of the table cannot be read honestly without it. Section 7 explains what it means.

6.6 All three runs side by side



Figure 7. Four test images. Columns left to right: visible input, real thermal, Run 0, Run A, Run B.

7. Analysis

7.1 Which run produced better outputs

Run B, by eye. Run A, by SSIM. The two disagree, and working out why is the most interesting part of this experiment, so we will not pick one and quietly drop the other.

Looking at Figure 7, both fine-tuned runs are doing the task and Run 0 is not. People are bright, foliage is dark, headlights no longer glow. Between A and B, Run B is consistently the better image. In row 1 Run B separates the pedestrians from the background and keeps the shape of the parked cars; Run A has a heavy bright halo around the centre that swallows the pavement. In row 2 Run B's figures have arms and legs; Run A's are soft blobs. In row 4 Run B keeps the branch structure of the trees, which is clearly present in the real thermal image; Run A has lost it under glow. Figure 4 shows Run A developing a speckled dotted texture across the foliage from epoch 3 onwards that never appears in Run B.

On SSIM, Run A scores 0.2829 and Run B scores 0.2579. The gap comes almost entirely from brightness. Run B's mean luminance error is -24.99 grey levels, meaning its images are systematically about a tenth of the full range too dark. SSIM penalises that through its luminance term even when the structure underneath is right. So the number is telling us something true, that Run B's absolute brightness calibration is worse, but it is not telling us Run A's pictures are better pictures.

7.2 When the difference first became visible

Epoch 3. At epochs 1 and 2 both runs produce dark, blobby images where the only thing you can reliably make out is that something bright is standing where a person is. At epoch 3 Run B starts resolving recognisable human figures and the vertical structure of the building columns, while Run A at the same checkpoint is still mostly dark with a broad glow. By epoch 5 the gap is obvious and the speckle artefact has appeared in Run A. The two runs are indistinguishable for the first two epochs and clearly different from the third onwards.

7.3 Why this happens, in terms of the schedule

First: the two loss values must not be compared

Run B's loss is lower at every step, and this is almost entirely an artefact. The loss is the mean squared error between the true noise epsilon and the model's prediction of it, averaged over randomly chosen timesteps. But the two runs draw their training examples from different forward processes, so they are not being asked the same question.

Under Run B's schedule, far more of the timestep range sits at very low alpha-bar, where x_t is almost exactly epsilon itself. When the input is nearly pure noise, predicting the noise is close to copying the input, and the error is small. Run B's loss is lower because a larger share of its questions are easy ones, not because it answers them better. Comparing 0.078 against 0.134 and concluding that Run B trained better would be a mistake. The only fair use of the loss curves is to check that each run on its own is descending and stable, which both are.

Second: why Run B's images are better anyway

The real mechanism is the terminal part of the schedule. Under Run A, sqrt(alpha-bar) at t = 1000 is 0.068, so about 7 percent of the target image amplitude is still present at the timestep that is supposed to be pure noise. The model is never shown a genuinely blank starting point. At high timesteps it can still faintly see the input it is conditioned on, and the cheapest way to reduce the loss is to learn a recolouring of what it can see rather than to generate thermal structure from nothing. That shortcut is exactly what Run A's outputs look like: visible-light texture carried through, speckle on the foliage where the original leaf detail was, and a global glow that is a brightness adjustment rather than a heat map.

Under Run B, sqrt(alpha-bar) at t = 1000 is 0.0018. There is nothing left to copy. The model is forced to produce the overall layout of a thermal image from pure noise plus the conditioning, which is the thing we actually want it to learn. That matches the observation that Run B is better at global structure and worse at absolute brightness: it has spent more of its training budget in the low-signal region where layout is decided, and less in the high-signal region where fine detail and exact grey levels are tuned.

Figure 6 is the direct visual confirmation of this. Half way through sampling, at t = 509, Run A has already laid down the whole scene and is refining it, while Run B is still working in noise and has committed to almost nothing. Run B is spending the second half of its reverse process doing the work Run A finished in the first half. The rescaling divisors in Table 4 say the same thing numerically: at the first capture Run B's latent sits 32 times further outside the decoder's range than Run A's, and the two only agree once the process is nearly over.

The same explanation accounts for the contrast numbers. Run A's contrast ratio is 1.623, meaning it is producing images with 62 percent more spread than the real thermal image, which is what a model that is exaggerating edges and texture would do. Run B's is 1.357, closer to the correct 1.0.

One honest caveat. This is a single run of each arm, on one dataset, with one exponent, on 2,000 training pairs and 2,500 steps. We did not repeat either run with a different seed, so we cannot separate a schedule effect from a lucky initialisation. The explanation above is consistent with everything we measured, but it is an explanation of one observation, not a proven general law about exponents above 1.

7.4 How Run 0 compares, and what it says about out-of-distribution fine-tuning

Run 0 has the best SSIM in the table, at 0.2954, and it is the only one of the three that is not doing the task at all. That contradiction needed explaining, so we ran a control: we scored the raw visible-light input against the thermal ground truth, with no model in the loop whatsoever.

What was scored	SSIM	Above the control
the visible input itself, no model	0.2591	-
Run 0 output	0.2603	+0.0012
Run A output	0.1808	-0.0783
Run B output	0.1963	-0.0628

Table 6. The control. These SSIM values are computed on the matched subset where all four were available, which is why they differ slightly from Table 5; the ordering and the conclusion are the same.

Run 0 scores 0.0012 above doing literally nothing. Its entire score comes from the scene geometry it copied out of the input photograph. SSIM here is measuring how much of the input survived, not how thermally correct the output is, and since Run 0's strategy is to leave the input mostly alone, it wins on that measure by not trying.

Both fine-tuned runs score below the control, and that is the expected and correct behaviour. They are no longer reproducing the input; they are producing a different image whose brightness has been reassigned according to temperature. Reassigning brightness necessarily destroys the pixel-level agreement with the input photograph, so SSIM has to fall. The drop is a sign that fine-tuning worked, not that it failed.

What this reveals about fine-tuning on an out-of-distribution dataset is a split result. 2,500 LoRA steps on 2,000 pairs was clearly enough to teach the appearance convention: hot is bright, people are the brightest thing in a night scene, vegetation is dark, glass and LED lights are dark. The model learned the grammar of a thermal image from a base that had never seen one. It was not enough to teach the physics. Neither run gets absolute temperature right. Run A is 11 grey levels too bright on average and Run B is 25 too dark, and neither gets the actual temperature of any specific surface correct, only its

rank. A model that had learned the physics would place a particular road surface at a particular grey level; ours place it somewhere plausible.

The last thing this shows is a methodological one that caught us out. If we had trusted the headline SSIM column and nothing else, we would have written that the pretrained model with no fine-tuning beat both fine-tuned runs, and that the modified schedule made things worse. Both statements are false and the images show they are false. The control row is what made that visible.

8. Takeaway

The exponent is not a cosmetic change. Raising alpha-bar to 2.35 moves the half-way point of corruption 148 timesteps earlier and stretches the log signal-to-noise curve by exactly the factor 2.35 in the low-signal region, which we were able to confirm numerically to three decimal places. Seeing the measured ratio come out at 2.35 when the algebra predicted 2.35 was the point where the transform stopped being an instruction in a handout and started being something we understood.

Choosing which quantity to raise mattered more than choosing the exponent. Our first plan used beta as theta. We checked what that implied before writing any training code and found it left 96 percent of the image intact at the final timestep, so Run B would have been trained on a forward process that barely adds noise. Running it would have produced a report full of confident conclusions about a broken experiment. Doing the arithmetic first cost an afternoon and saved the project.

Training loss across two different schedules is not a comparable number. Run B's loss is 42 percent lower than Run A's and that tells us nothing about which model is better, because the two runs are answering different questions. We nearly reported it as a result.

A metric can agree with a sensible definition and still be useless. SSIM is a reasonable choice for this task and we would choose it again, but on this data it ranked the model that does nothing above both models that do the task. The control run, scoring the input against the target with no model at all, was the cheapest and most valuable thing we did, and it is a habit we will keep.

On the mechanism itself: the benefit we observed from the powered schedule came from removing the leftover signal at the final timestep. The original schedule lets the model see about 7 percent of the answer at the point where it should see nothing, and the model exploits that by learning to recolour rather than to generate. Closing that gap forced it to actually build a thermal image. That is a specific, checkable reason, and it is the kind of reason we would not have found by only looking at scores.

Finally, almost all of the engineering difficulty was not in the diffusion mathematics. It was in the splitting of a dataset of video frames so that the test set was genuinely unseen, in keeping two runs identical in every respect but one, and in proving afterwards that they had been. The checksum on the schedule array is nine lines of code and it is the only reason we can state with confidence that this report is about the thing we think it is about.

9. Contribution Statement

The components below were divided among the four members of Group 38. Each member has signed against the work they were responsible for.

Member	Component	Roll no.	Signature
	Task and model selection, schedule analysis and verification, Phase 1 declaration		
	Dataset preparation, scene-disjoint splitting, caching pipeline		
	Code generation pipeline, training script patches, both fine-tuning runs		
	Inference, checkpoint sampling, trajectories, evaluation and report		

Signed:

Name	Name	Name	Name

Appendix - Files submitted with this report

Everything listed here is in the submitted project folder. The paths are relative to the project root.

A.1 Code

Path	What it is
main.py	runs all fifteen stages end to end
scripts/schedule.py	the power transform, bounds check, checksums; POWER_EXPONENT is on line 15
scripts/make_finetune.py	generates finetune.py from the untouched baseline
scripts/finetune.py	the training script, 104 # ADDED and 12 # MODIFIED lines
scripts/prepare_llvip.py	scene-disjoint split and image cache
scripts/llvip_data.py	the dataset class
scripts/infer.py	image generation for Run 0, A and B
scripts/trajectory.py	the x_t captures
scripts/evaluate.py	the scoring
scripts/epoch_losses.py	collapses per-step loss into per-epoch loss
scripts/make_diff.py	writes report/code_diff.txt; a script because PowerShell aliases diff to Compare-Object
scripts/check_env.py	environment check and benchmark
baseline/ train_instruct_pix2pix_ORIGINAL.py	the unmodified upstream file, for the diff
README.md	exact commands, seed and software environment
requirements.txt	pinned library versions

A.2 Fine-tuning logs

Path	What it is
logs/runA_epoch_loss.csv	epoch, train_loss - the required CSV for Run A
logs/runB_epoch_loss.csv	epoch, train_loss - the required CSV for Run B
logs/runA_loss.csv, logs/runB_loss.csv	per-step loss, used for Figure 3
logs/env.json	Python and library versions, GPU, benchmark
outputs/runA/ckpt-500 ... ckpt-2500	4 samples each, 5 checkpoints, Run A
outputs/runB/ckpt-500 ... ckpt-2500	4 samples each, 5 checkpoints, Run B

A.3 Inference trajectories

Path	What it is
trajectories/runA/	15 images, 3 samples x 5 timesteps, named run_sample_step_t
trajectories/runB/	15 images, same structure
trajectories/run*/_trajectory_info.json	timestep index, checksum and display scale for every file

A.4 Schedule evidence

Path	What it is
schedules/schedule_runA.csv	all 1000 values of beta, alpha, alpha-bar and SNR for Run A
schedules/schedule_runB.csv	the same for Run B
schedules/checksums.txt	the two expected checksums and the exponent
outputs/runA/schedule/, outputs/runB/schedule/	what each training run actually used

A.5 Results

Path	What it is
outputs/run0/	Run 0 outputs on the 100 test images
outputs/runA/final/, outputs/runB/final/	final outputs on the 100 test images
report/metrics.csv	the scores in Table 5
report/per_image_run0.csv, _runa.csv, _runb.csv	per-image scores
report/figures/	all figures used in this report
report/code_diff.txt	the full line-by-line diff against the baseline

A.6 Software environment

Component	Version
Python	3.12.10
Operating system	Windows 11 (10.0.26300)
torch	2.11.0+cu128
diffusers	0.31.0
transformers	4.46.3
accelerate	1.1.1
peft	0.13.2
GPU	NVIDIA GeForce RTX 5050 Laptop GPU, compute capability sm_120, 7.96 GiB
Random seed	20261026